

AI-Readable Architecture: Yapay Zekâ Çağında Yazılım Mimarisinin Yeniden Tanımlanması

Ömer Faruk Yavuz

Temmuz 2026

Giriş

Yapay zekâ destekli yazılım geliştirme çağında bazı mimari pratikler yalnızca “temiz kod” yaklaşımının uzantısı olarak görülemez. Bu pratikler artık daha geniş bir işlev üstlenmektedir: sistemi yalnızca insan geliştiriciler tarafından değil, yapay zekâ araçları tarafından da okunabilir, anlaşılabilir ve güvenli biçimde değiştirilebilir hale getirmek. Bu çalışma, söz konusu yaklaşımı ifade etmek üzere **AI-readable architecture** kavramını incelemektedir.

AI-readable architecture, bir kod tabanının sınırlarının, veri akışlarının, testlerinin, isimlendirmelerinin, sorumluluklarının ve bağımlılıklarının hem insan hem de makine tarafından doğru yorumlanabilecek biçimde tasarlanmasıdır. Bu yaklaşımda amaç yalnızca çalışan kod üretmek değildir; amaç, insan ve yapay zekânın aynı sistem üzerinde birlikte güvenilir biçimde akıl yürütmesini sağlamaktır.

Çalışmanın temel savı şudur: Yapay zekânın kod üretimindeki başarısı yalnızca modelin kapasitesine değil, üzerinde çalıştığı kod tabanının yapısal niteliklerine de bağlıdır. Bu nedenle yapay zekâ çağında mimari kalite, rekabetçi bir değişken haline gelmiştir ve önümüzdeki dönemde kod tabanları iki kutup arasında ayrışacaktır: yapay zekâ ile ortak çalışmaya uygun tasarlanmış sistemler ile teknik borcun temel işleyiş biçimine dönüştüğü sistemler.

Yapay Zekânın Örüntü Tamamlama Doğası

Yapay zekâ sistemleri büyük ölçüde olasılık tabanlı örüntü tamamlama mekanizmasıyla çalışır; verilen bağlamdan hareketle bir sonraki en olası yapıyı üretmeye çalışırlar. Bu mekanizmanın mimari açıdan önemli bir sonucu vardır: kod tabanı tutarlıysa doğru sonraki adımı tahmin etmek kolaylaşır, çünkü tekrar eden yapılar anlamlıdır, isimlendirmeler karardır, sınırlar açıktır ve testler davranışı tarif eder.

Kod tabanı kaotikse, olası doğru yolların sayısı artar. Aynı problem için birden fazla çözüm biçimi mevcutsa, model hangi yolun geçerli olduğunu ayırt etmekte zorlanır. Bu ilişki şu şekilde özetlenebilir:

Tutarlı kod tabanı, yapay zekânın olasılık uzayını daraltır; kaotik kod tabanı ise bu uzayı genişleterek belirsizliği artırır.

Dolayısıyla yapay zekânın güvenilirliği, yalnızca modelin gücünün değil, sistemin sunduğu örüntü kalitesinin de bir fonksiyonudur.

Yapay Zekânın Etkinliğini Artıran Mimari Özellikler

Yapay zekâ, sistemi anlamak için mevcut kodun isimlendirmelerine, dosya sınırlarına, sorumluluk dağılımına, testlerine ve tekrar eden örüntülerine bakar. Bu nedenle iyi mimari, yalnızca insan geliştirici için değil, yapay zekâ için de bir yön bulma haritası işlevi görür.

Açık Mimari

Açık mimari, sistem parçalarının işlevlerinin ve birbirleriyle kurdukları ilişkilerin kolayca anlaşılabilmesidir. Hangi dosyanın arayüzden, hangisinin durum yönetiminden, hangisinin dış sistem iletişiminden, hangisinin iş kuralından sorumlu olduğu netse, yapay zekâ da değişikliği doğru konuma yerleştirebilir.

Açık mimarinin yokluğunda model tipik bir hata üretir: yeni kodu çalışabileceği ilk yere yerleştirir, fakat mimari açıdan doğru yere yerleştirmez. Bu davranış kısa vadede işlevsel görünür; ancak uzun vadede teknik borcu artırır.

İyi Tanımlanmış Sınırlar

Sınır (boundary), sistem parçaları arasındaki ayrım çizgisidir: arayüz ile iş mantığı arasında, dış sistem istemcisi ile durum yönetimi arasında, bileşen ile servis arasında sınırlar bulunmalıdır.

İyi tanımlanmış sınırlar mevcut olduğunda model şu soruları daha isabetli yanıtlar: kod nereye yazılmalı, hangi dosya hangi sorumluluğu taşıyor, hangi değişiklik hangi alanı etkilemeli, hangi alanlara dokunulmamalı. Sınırların yokluğunda model her şeyi her yere yerleştirebilir; bu da sistemi zamanla daha fazla karıştırır.

İsimlendirme

İsimlendirme, yazılım sisteminin dilidir. İyi adlandırılmış değişkenler, fonksiyonlar, bileşenler ve servisler, model için güçlü anlamsal sinyaller üretir. Belirsiz bir fonksiyon adı yalnızca bir işlemin varlığını bildirirken, bağlamı ve amacı içeren bir ad; fonksiyonun ait olduğu alan kavramını, kullanım amacını ve muhtemel etki alanını da iletir.

Bu nedenle iyi isimlendirme yalnızca okunabilirlik meselesi değildir; yapay zekâyı doğru bağlam aktarmanın temel yöntemlerinden biridir.

Modülerlik

Modülerlik, sistemin küçük, anlamlı ve bağımsız parçalara ayrılmasıdır. Her modülün net bir sorumluluğu olduğunda, model o parçayı daha kolay anlayabilir, değiştirebilir ve test edebilir.

Büyük ve karmaşık dosyalar model için zorlayıcıdır; çünkü model arayüz detayını, durum değişimini, dış sistem çağrısını, hata yönetimini ve iş kuralını aynı anda izlemek zorunda kalır. Modüler yapıda ise model daha dar bir problem alanına odaklanır. Bu durumun kritik bir sonucu vardır:

Modülerlik, yapay zekânın bağlam yükünü azaltır.

Modelin bağlam penceresi sınırlı olduğundan, kod ne kadar iyi bölünmüşse doğru parçanın anlaşılması o kadar kolaylaşır.

Test Kapsamı

Test kapsamı, sistem davranışlarının ne ölçüde testlerle güvence altına alındığını gösterir. Yapay zekâ açısından testler yalnızca hata yakalama aracı değildir; sistemin beklenen davranışını tarif eden canlı dokümantasyondur.

Nitelikli testler mevcutsa model şunları görebilir: sistem ne yapmalı, hangi davranış korunmalı, sınır durumları nelerdir, değişiklikten sonra ne bozuldu, önerilen çözüm gerçekten güvenli mi. Testlerin zayıf olduğu koşullarda model kod üretir; fakat ürettiği kodun sistemi bozup bozmadığını sağlıklı biçimde değerlendiremez.

Deterministik Akış

Deterministik akış, aynı girdi verildiğinde sistemin aynı sonucu üretmesidir; akışların rastgele yan etkilere, gizli duruma, sıra bağımlılıklarına veya belirsiz koşullara bağlı olmamasıdır.

Yapay zekâ deterministik sistemlerde daha iyi çalışır; çünkü neden–sonuç ilişkisini daha net kurabilir. Bir eylemin hangi durum dönüşümünü tetiklediği, hangi verinin nereden okunduğu ve arayüzün nasıl güncellendiği izlenebilir olduğunda model doğru analiz yapabilir. Buna karşılık sistemde kontrolsüz yan etkiler mevcutsa, model şu temel soruda zorlanır: bu değişiklik gerçekten yalnızca bu bölgeyi mi etkiler, yoksa başka bir yerde sessizce başka bir davranış mı tetikler?

Yapay Zekânın Güçlü Olduğu Alanlar

Yapay zekâ, özellikle düzenli, tekrar eden ve açık örüntülere sahip kod tabanlarında yüksek performans gösterir. Bu güçlü alanlar sekiz başlık altında incelenebilir.

Örüntü Devamlılığı

Örüntü devamlılığı (pattern continuation), modelin mevcut kod tabanındaki tekrar eden yapıları anlayıp aynı yapının devamını üretebilmesidir. Örneğin durum yönetimi akışının bileşenleri tutarlı biçimde yazılmışsa, model yeni bir özellik için benzer akışı doğru biçimde oluşturabilir.

Ancak bu mekanizmanın sağlıklı çalışması için mevcut örüntünün gerçekten doğru olması gerekir. Sistemde kötü örüntüler tekrar ediyorsa, model bu yapıları da aynı güvenle sürdürür. Dolayısıyla örüntü devamlılığı yalnızca iyi mimaride avantajdır; kötü mimaride teknik borcu çoğaltan bir mekanizmaya dönüşür.

Yerel Tutarlılık

Yerel tutarlılık (local consistency), modelin belirli bir dosya veya modül içindeki mevcut yapıya uyumlu kod üretebilmesidir. Bir modül içinde belirli bir isimlendirme standardı, hata yönetimi biçimi veya yardımcı yapı kullanımı mevcutsa, model yeni eklemeleri genellikle buna uygun biçimde yapar.

Bu güçlü bir özelliktir; fakat sistemin genel mimarisi tutarsızsa yerel tutarlılık yeterli olmaz. Model, dar bölgede doğru görünen ancak sistem genelinde yanlış olan çözümler üretebilir.

Tekrarlayan Refactor İşlemleri

Model, tekrar eden yeniden düzenleme işlerinde oldukça etkilidir: dağınık sabit değerleri merkezi yapılara taşımak, tekrar eden biçimlendirme kodlarını yardımcı fonksiyonlara çıkarmak veya benzer bileşen yapılarını tek tipe indirgemek gibi işler bu kapsamdadır. Bu tür işler insan için zaman alıcı ve yorucudur; model burada belirgin hız kazandırır.

Ancak hedef mimarinin insan tarafından belirlenmesi gerekliliği devam eder. Model, iyi başlatılmış bir refactor yönünü sürdürebilir; fakat hangi mimari yönün doğru olduğuna her zaman kendi başına karar veremez.

Test Üretimi

Model, test üretiminde güçlüdür; özellikle fonksiyonların girdi-çıkı davranışı net, bileşen arayüzleri açık ve veri sözleşmeleri belirliyse anlamlı test senaryoları oluşturabilir. Ancak kritik bir sınırlama mevcuttur:

Model test yazabilir; fakat hangi davranışın iş açısından gerçekten doğru olduğunu her zaman bilemez.

Bu nedenle test üretiminde alan bilgisinin, iş kurallarının ve beklenen davranışın açıkça tanımlanmış olması belirleyicidir.

Kalıp Kod Üretiminin Azaltılması

Tekrar eden mekanik kodların (boilerplate) üretimi veya azaltılması modelin verimli olduğu alanlardandır. Yeni bir özellik için gerekli iskelet yapıların oluşturulması çoğu zaman mekanik bir iştir ve model bu tür işleri hızla tamamlayabilir. Bu, geliştiricinin enerjisini daha değerli alanlara — doğru iş mantığı, doğru sınırlar, doğru durum modeli, doğru kullanıcı akışı ve doğru hata yönetimi — yöneltmesini sağlar.

Geçiş (Migration) Desteği

Model, çerçeve veya mimari geçiş süreçlerinde de yardımcı olabilir: eski bir yapıyı yeni bir örüntüye taşımak, kullanımdan kaldırılmış arayüzleri güncellemek veya bileşen yapısını yeni standarda uyarlamak gibi. Ancak eski sistemde gömülü iş mantığı, geçici çözümler ve üretim ortamına özgü sınır durumları mevcutsa, model bunları gözden kaçırabilir. Bu nedenle geçiş süreçlerinde model hızlandırıcıdır; fakat mimari karar verici olarak tek başına yeterli değildir.

Tip Yayılımı

Tipli sistemlerde model daha güçlü çalışır. Tipler, sistemin anlamını görünür kılar: bir veri modeli değiştiğinde ilgili tiplerin, arayüz tanımlarının, fonksiyon imzalarının ve kullanım noktalarının güncellenmesi gerekir ve model bu yayılım işlerinde oldukça etkilidir.

Tipler, model için yalnızca derleyici bilgisi değildir; alan kavramlarının yapısal haritasıdır:

Tip sistemi, yapay zekâya sistemin geometrisini gösteren temel araçlardan biridir.

Büyük Ölçekli Yeniden Adlandırma

Bir kavramın adı değiştirildiğinde bu değişikliğin dosya adlarına, fonksiyonlara, bileşenlere, testlere ve dokümantasyona yansıtılması gerekir. Bu tür işler dikkat gerektirir ve insan için yorucudur; model burada ciddi hız kazandırır. Ancak temel ilke değişmez: kavramın neye dönüşmesi gerektiğine insan karar verir; model bu dönüşümü sistem geneline yayar.

Yapay Zekânın Zorlandığı Alanlar

Yapay zekâ her alanda eşit derecede güçlü değildir. Özellikle kodda açıkça görünmeyen tarihsel, organizasyonel veya alana özgü bilgilerde belirgin biçimde zorlanır.

Örtük İş Mantığı

Örtük iş mantığı, kodda açıkça ifade edilmeyen fakat sistemin davranışını belirleyen kurallardır: belirli bir alanın belirli müşteri grupları için farklı çalışması, bazı kullanıcıların özel yetkilere sahip olması veya eski verilerin farklı yorumlanması gibi. Bu kurallar kodda açık değilse model onları göremez ve teknik açıdan doğru görünen fakat iş açısından yanlış olan çözümler üretebilir.

Tarihsel Kararların Gerekçesi

Bir sistemdeki bazı kararlar, geçmişteki özel koşullar nedeniyle alınmıştır: bir uç noktanın alışılmadık davranışı eski bir dış entegrasyonu korumak için tasarlanmış, bir veri modelindeki tuhaf alan geçmiş bir veri taşıma probleminden kalmış olabilir. Model mevcut kodu görebilir; fakat kararın tarihsel gerekçesini bilemez. Bu nedenle dokümantasyon, sürüm geçmişi, kayıt sistemleri ve ekip hafızası değerini korumaktadır.

Geçici Çözümlerin Varlık Nedeni

Bazı kodlar ilk bakışta hatalı görünür; fakat üretim ortamında yaşanmış özel bir problemi çözmek için yazılmıştır. Rastgele görünen bir zaman aşımı değeri belirli bir üçüncü taraf servisin davranışına göre seçilmiş, gereksiz görünen bir koşul bloğu eski bir veri formatını desteklemek için eklenmiş olabilir.

Model bu tür yapıları kolayca silebilir veya sadeleştirebilir; bu da gerçek üretim hatalarına yol açar. Dolayısıyla geçici çözümler mümkün olduğunca açıklama, test veya dokümantasyonla desteklenmelidir.

Organizasyonel Bağlam

Bazı teknik kararlar yalnızca teknik gerekçelerle alınmaz. Kurum içi ekip yapısı, sahiplik sınırları, müşteri ilişkileri, yönetsel öncelikler, teslim takvimleri veya güvenlik politikaları da mimariyi etkiler. Model bu bağlamı doğal olarak taşımaz; bu nedenle dışarıdan bakıldığında tuhaf görünen bazı mimari kararlar, organizasyonel olarak zorunlu olabilir.

Üretim Ortamı Sınır Durumları

Üretim ortamı sınır durumları, gerçek kullanıcıların, gerçek verinin ve gerçek sistem koşullarının ortaya çıkardığı özel durumlardır: eksik geçmiş veriler, bozuk taşıma kayıtları, yavaş üçüncü taraf servisler, beklenmeyen ağ kesintileri, ardışık hızlı kullanıcı etkileşimleri, farklı saat dilimi davranışları veya eski istemci ortamları. Bu durumlar çoğu zaman yerel geliştirme ortamında görünmez ve testlerde yakalanmamış olabilir. Model bu sınır durumlarını ancak açıkça verildiğinde veya testlerle görünür kılındığında dikkate alabilir.

Örtük Varsayımlar

Örtük varsayımlar, sistemde açıkça yazılmamış fakat herkesin doğru kabul ettiği kabullerdir: her kaydın mutlaka bir sahibi olduğu, her dosyanın belirli bir formatta geldiği, her sunucu yanıtının belirli alanları içerdiği veya belirli bir tanımlayıcının küresel olarak benzersiz olduğu gibi. Bu varsayımlar kodda görünür değilse model onları fark etmeyebilir. Teknik borç arttıkça örtük varsayım sayısı da artar ve modelin güvenilirliği buna paralel olarak düşer.

Alan Gerçekliği

Alan gerçekliği (domain truth), sistemin bağlı olduğu gerçek dünyanın bilgisidir ve her zaman koddan çıkarılamaz. Bir ölçümün klinik anlamı, bir medya varlığının kullanım bağlamı veya bir işlem türünün hukuki karşılığı bu kapsama girer. Model kodu okuyabilir; fakat alanın gerçekliğini her zaman koddan çıkaramaz. Bu nedenle alan gerçekliği açıkça modellenmeli, test edilmeli ve dokümanite edilmelidir.

Teknik Borç ile Zorlanma Alanlarının Genişlemesi

Teknik borç arttıkça sistemde açık bilgi azalır, gizli bilgi artar: sınırlar zayıflar, akışlar bulanıklaşır, testlere güven azalır ve kararların gerekçesi kaybolur. Bu koşullarda yapay zekânın güçlü olduğu alanların etkisi daralır; zorlandığı alanların etkisi genişler.

Bu kayma şu şekilde ifade edilebilir:

Teknik borç arttıkça sistem; açık örüntülerden gizli varsayımlara, deterministik akışlardan belirsiz yan etkilere, test edilebilir davranışlardan üretim ortamı sürprizlerine doğru kayar.

Bu nedenle model, teknik borçlu sistemlerde başlangıçta hızlı görünse bile zamanla daha fazla hata üretmeye başlar.

İki Tip Kod Tabanı: Yapısal Ayrışma

Yapay zekâ destekli geliştirme yaygınlaştıkça, kod tabanları kalite açısından daha keskin biçimde ayrışacaktır. Bu ayrışma iki ideal tip üzerinden tanımlanabilir.

AI-Native Kod Tabanı

AI-native kod tabanı, yapay zekâ ile ortak çalışmaya uygun biçimde tasarlanmış sistemdir. Bu tür sistemlerin ortak özellikleri şunlardır:

- net sınırlar,
- güçlü ve güvenilir testler,
- deterministik mimari,
- güncel dokümantasyon,
- tipli sistemler,
- gözlemlenebilir akışlar,
- açık alan modelleri,
- tutarlı isimlendirme standardı,
- izole edilebilir modüller,
- güvenilir sürekli entegrasyon ve dağıtım akışları.

Bu yapılarda küçük ekipler, yapay zekâ desteğiyle çok büyük çıktılar üretebilir; çünkü model yalnızca kod yazmakla kalmaz, mevcut mimariyle uyumlu biçimde yeniden düzenleme yapabilir, test üretebilir, geçiş süreçlerini destekleyebilir ve sistem genelinde tutarlılık sağlayabilir. Bu koşullarda beş kişilik bir ekip, yapay zekâ desteğiyle klasik dönemin çok daha kalabalık ekiplerinin çıktısına yaklaşabilir veya onu aşabilir. Buradaki belirleyici fark yalnızca yapay zekâ kullanmak değildir; kod tabanının yapay zekâ ile ortak çalışmaya uygun olmasıdır.

Teknik Borç Bataklığı

Teknik borç bataklığı (technical debt swamp), teknik borcun sistemin temel işleyiş biçimine dönüştüğü kod tabanıdır. Bu sistemlerde şu problemler yaygındır:

- düğümlenmiş eski kod yapıları,
- örtük mantık,
- rastgele bağlaşım,
- güven vermeyen testler,
- belgelenmemiş akışlar,
- kırılğan dağıtım süreçleri,
- belirsiz sahiplik,
- eski varsayımlar,

- gerekçesi bilinmeyen geçici çözümler,
- üretim ortamında öğrenilmiş fakat kodda belgelenmemiş davranışlar.

Bu tür sistemlerde model sistemin gerçek davranışını göremez; kodda görünen yapı ile üretimde çalışan gerçek mantık farklılaşmıştır. Daha tehlikeli olan, modelin yanlış güven üretebilmesidir: yanlış çözümü doğruymuş gibi, hızlı ve ikna edici biçimde sunabilir. Bu davranış deneyimsiz geliştirici davranışına benzer; ancak hız ve ikna edicilik nedeniyle bazı durumlarda daha risklidir. Bu nedenle teknik borç bataklığında yapay zekâ kullanımı özel dikkat gerektirir; aksi halde sistem daha hızlı gelişmez, daha hızlı karışır.

Örtük Uzman Hafızasından Makineyle Paylaşılabilir Mimariye

Geleneksel yazılım geliştirme döneminde teknik borç, bazı durumlarda yönetilebilir görünüyordu. Bunun önemli nedenlerinden biri, sistemi yazan veya uzun süredir bakımını üstlenen kıdemli mühendislerin sistemin bağımlılık yapısını zihinsel olarak taşıyabilmesiydi. Kod tabanı dışarıdan karmaşık görünse bile, belirli kişiler sistemin tarihini, geçici çözümlerini, sınır durumlarını ve kritik akışlarını biliyordu.

Yapay zekâ çağında bu model yetersizdir; çünkü üretkenlik artık yalnızca insanın sistemi anlamasına değil, insan ve yapay zekânın birlikte akıl yürütebilmesine bağlıdır. Bu nedenle sistem bilgisi yalnızca kıdemli mühendisin zihninde kalmamalı; mimariye, tiplere, testlere, dokümantasyona, gözlemlenebilir akışlara ve açık sözleşmelere taşınmalıdır.

Bu geçiş şu şekilde özetlenebilir:

Kıdemli mühendisin zihninde taşınan örtük sistem bilgisi, makine tarafından da okunabilir açık mimariye dönüştürülmelidir.

Yapay Zekâ Çağında Değer Kazanan Mimari Nitelikler

Yukarıdaki analiz ışığında, yapay zekâ çağında altı mimari nitelik önceki dönemlere kıyasla belirgin biçimde değer kazanmıştır.

Açık (Explicit) Mimari

Açık mimari, sistemin mimari yapısının yalnızca ekip içinde sözlü olarak bilinmesi değil; kod organizasyonu, dokümantasyon, dosya yapısı ve örüntüler aracılığıyla görünür olmasıdır. Model, sistemin nereye ve nasıl genişletileceğini mevcut yapıdan okuyabildiği için açık mimari doğrudan üretim kalitesine yansır.

Tipli Sınırlar

Tipli sınırlar, sistemin katmanları ve modülleri arasındaki veri geçişlerinin açık tiplerle tanımlanmasıdır. Tipler; verinin biçimini, zorunlu alanları ve alan ilişkilerini görünür kılar. Böylece yalnızca derleme güvenliği değil, sistemin anlam haritasının makine tarafından okunabilirliği de sağlanır.

Deterministik Akışlar

Aynı girdi ve aynı koşullar altında aynı sonucu üreten akışlar, neden–sonuç ilişkisini izlenebilir kılar. Bir eylemin hangi dönüşümü tetiklediği ve verinin hangi yoldan aktığı belirli olduğunda, model değişikliklerin etkisini daha isabetli öngörebilir.

Güçlü Sözleşmeler

Güçlü sözleşmeler (contracts), sistemin farklı parçaları arasındaki anlaşmaların açıkça tanımlanmasıdır: istek–yanıt şemaları, bileşen arayüzleri, olay yükleri, durum modelleri ve test beklentileri birer sözleşme olarak görülebilir. Sözleşmeler güçlü olduğunda model, değişikliklerin sınırlarını daha doğru kavrar.

Modülerlik

Sistemi küçük ve anlamlı parçalara ayıran modülerlik, modelin bağlam yükünü azaltır. Büyük ve karmaşık dosyalarda model çok sayıda sorumluluğu aynı anda izlemek zorunda kalırken, modüler sistemlerde her parça daha net bir anlam taşır.

Gözlemlenebilirlik

Gözlemlenebilirlik, sistemin çalışma zamanı davranışının izlenebilir olmasıdır: kayıtlar, metrikler, izler, hata ayıklama yüzeyleri ve kontrollü test ortamları bu kapsama girer. Model yalnızca statik kodu değil, sistemin davranışsal sinyallerini de kullanabildiğinde daha güvenilir öneriler üretir.

Anlam Mimarisi ve Sistem Akışı

AI-readable architecture kavramı, iki tamamlayıcı düşünceyle doğrudan ilişkilidir: anlam mimarisi ve sistem akışı.

Anlam mimarisi, sistemdeki kavramların, sorumlulukların, ilişkilerin ve kararların okunabilir bir yapı içinde düzenlenmesidir. Sistem akışı ise verinin, olayların, durum değişimlerinin ve kullanıcı etkileşimlerinin hangi yoldan ilerlediğini açık hale getirmektir. Bu iki unsur birleştğinde ortaya çıkan şey şudur:

Hem insanın hem yapay zekânın üzerinde düşünebildiği, izleyebildiği ve güvenle değiştirebildiği bir yazılım yapısı.

Bu çerçevede, gelecekte en değerli mühendislik rollerinden biri yalnızca kod yazan kişi olmayacaktır. Daha değerli rol, yapay zekânın azami verimle çalışabileceği sistemleri tasarlayan kişidir. Bu mühendis; sistemi modüler hale getirir, alan kavramlarını açıklar, sınırları net çizer, testleri davranış dokümantasyonu gibi tasarlar, akışları gözlemlenebilir kılar, örtük varsayımları açık hale getirir ve teknik borcu yalnızca temizlik değil, bir akıl yürütme problemi olarak ele alır. Bu yaklaşım klasik anlamda yalnızca yeniden düzenleme değildir; sistemin anlam mimarisini kurmaktır.

Sonuç

Yapay zekâ yazılım geliştirmeyi hızlandırır; fakat bu hızın kalitesi ve sürdürülebilirliği, kod tabanının yapısal niteliklerine bağlıdır. AI-readable architecture mevcut olduğunda yapay zekâ güçlü bir kaldıraç etkisi üretir: örüntüleri sürdürür, test yazar, yeniden düzenleme yapar, geçiş süreçlerini destekler, tipleri yayar ve tekrar eden işleri azaltır. Buna karşılık teknik borç bataklığında model aynı güvenilirlikle çalışamaz; örtük iş mantığı, tarihsel kararlar, üretim ortamı sınır durumları, örtük varsayımlar ve alan gerçekliği eksikliği modeli sistematik biçimde yanıltır.

Bu nedenle yapay zekâ çağında teknik borç daha pahalıdır ve mimari düşünce daha az değil, daha fazla önem kazanmıştır. Mesele artık yalnızca insanın kodu anlayıp anlayamaması değil; insan ve yapay zekânın aynı sistem üzerinde birlikte akıl yürütüp yürütemediğidir.

Bu çalışmanın temel sonucu şu şekilde özetlenebilir:

Geleceğin güçlü kod tabanları yalnızca insan tarafından okunabilir (human-readable) değil, aynı zamanda makine tarafından okunabilir (AI-readable) olacaktır.